

Embedded Data Structures and IoT Communication Protocols

Enums and Scoped Enums, Struct Padding and Alignment, Packed Data, and a Comparative Study of HTTP, MQTT, and ESP-NOW

Report Title	Embedded Data Structures and IoT Communication Protocols
Program	IoT Training Program
Scope	Section 1: structs, enums, alignment, padding, packed data and unions. Section 2: HTTP, MQTT and ESP-NOW compared for IoT.
Date	3/8/2026
Prepared By	Malak Alsharqawi
Organization	Cyber Robot

Internal Training Documentation

Contents

How to Read This Report.....	5
Section 1: Structs, Enums, and Unpacking Data.....	6
1.0 Why Data Layout Is a Design Decision, Not a Detail	6
1.1 Enums: Giving Names to Integers.....	6
1.1.1 How Values Are Assigned	7
1.1.2 Underlying Type and Size	7
1.2 The Two Kinds of Enumeration in C++.....	8
1.2.1 The C-style (Unscoped) Enum and Its Two Structural Problems	8
1.2.2 The Scoped Enum (enum class)	8
1.2.3 Side-by-Side Comparison.....	9
1.3 Why enum class Is Safer for State Machines.....	9
1.3.1 Reason One: Several State Machines Can Coexist	9
1.3.2 Reason Two: A State Cannot Be Compared Against a Raw Number	10
1.3.3 Reason Three: Two Different State Types Cannot Be Compared By Accident.....	10
1.3.4 Reason Four: Arithmetic on States Becomes Impossible	10
1.3.5 Reason Five: Predictable Size and Clean Headers	10
1.3.6 Reason Six: The Compiler Can Check That Every State Is Handled	11
1.3.7 What enum class Does Not Do	11
1.3.8 Worked Example: A Motor State Machine.....	11
1.3.9 Best Practice Summary.....	12
1.4 Struct Padding and Memory Alignment	12
1.4.1 Four Definitions Everything Else Depends On	12
1.4.2 Why Alignment Exists At All.....	13
1.4.3 Padding Between Members	13
1.4.4 The Alignment of the Struct Itself	13
1.4.5 End Padding, and Why Arrays Cause It.....	14
1.4.6 How Member Order Changes RAM Usage	14
1.4.7 The ABI Decides the Rules, Not the CPU	15
1.4.8 Tools for Inspecting Layout	15
1.5 What __attribute__((packed)) Does.....	16
1.5.1 What Packing Does Not Do.....	16
1.5.2 The Cost: Unaligned Members	17
1.5.3 The ESP32-Specific Picture	17
1.5.4 When to Use Packed, and When Not To	17
1.5.5 Worked Example: An ESP-NOW Telemetry Packet	17
1.5.6 The More Portable Alternative: Explicit Serialisation	18
1.6 Unpacking Data: Unions and Byte Views.....	18
1.6.1 The Three Reasons Unions Appear in Embedded Code	19

1.6.2 Two Caveats Worth Knowing Before Relying On This	19
1.6.3 Struct versus Union	20
1.7 Key Terms at a Glance	20
1.8 Section 1 Recap	20
Section 2: IoT Communication Protocols (HTTP, MQTT, and ESP-NOW)	22
2.0 Where These Protocols Sit	22
2.1 HTTP (HyperText Transfer Protocol).....	22
2.1.1 How It Works	23
2.1.2 Overhead, and the Polling Problem	23
2.1.3 Where HTTP Is the Right Answer on an ESP32	23
2.1.4 Advantages and Disadvantages	24
2.2 MQTT (Message Queuing Telemetry Transport).....	24
2.2.1 Architecture and Topics.....	24
2.2.2 Quality of Service.....	24
2.2.3 The Features That Have No HTTP Equivalent.....	25
2.2.4 What MQTT Costs.....	25
2.2.5 Where MQTT Is the Right Answer on an ESP32	25
2.2.6 Advantages and Disadvantages	26
2.3 ESP-NOW (Espressif Connectionless Protocol).....	26
2.3.1 How It Works, and the Numbers That Constrain It	26
2.3.2 Two Points the Common Summaries Get Wrong.....	27
2.3.3 Where ESP-NOW Is the Right Answer	27
2.3.4 Advantages and Disadvantages	27
2.4 Master Comparison Table	29
2.5 Reading the Differences: Overhead, Latency, and Power	29
2.5.1 HTTP versus MQTT	30
2.5.2 Why ESP-NOW Latency Is In a Different Category	30
2.6 Which One Should I Use?	30
2.6.1 Choose HTTP When	30
2.6.2 Choose MQTT When.....	30
2.6.3 Choose ESP-NOW When.....	31
2.6.4 A Short Decision Path	31
2.7 The Hybrid Pattern, Which Is Usually the Real Answer.....	31
2.8 Applied to the Motor Protection and Interlock Projects.....	32
2.9 Common Mistakes Checklist.....	32
2.10 Section 2 Recap	33
References	34
Primary documentation.....	34
Protocol references and articles.....	34

Memory layout, structs, unions and enums..... 34

Companion report**Error! Bookmark not defined.**

How to Read This Report

This report is a follow-on to the ESP32 Ecosystem, Communication Protocols, and Power Sizing report and keeps the same conventions. Every acronym is spelled out on first use. Every concept opens with a plain-language explanation or an everyday analogy before the formal engineering definition. Shaded boxes carry one of four labels: **In plain terms** for an intuitive restatement, **Analogy** for a comparison, **Why this matters** for the practical consequence, and **Fact check** or **Watch for this** where a widely repeated claim needs qualifying against primary documentation.

Code listings are illustrative and written for the Arduino and ESP-IDF environments on the ESP32. Numeric limits quoted for ESP-NOW, alignment behaviour, and compiler warnings are taken from Espressif's official documentation, the OASIS MQTT specification, HiveMQ's protocol documentation, and cplusplus, all listed in the References section. Where a claim is platform-specific it is stated as such rather than presented as a universal rule, since a great deal of embedded advice on the open web is accurate for one architecture and wrong for another.

The two sections are deliberately connected. Section 1 explains how a struct is laid out in memory and how to control that layout. Section 2 explains the protocols that carry those bytes between devices. The link between them is ESP-NOW, which transmits raw application bytes with no serialisation layer of its own, so the layout decisions of Section 1 become the wire format of Section 2.

Section 1: Structs, Enums, and Unpacking Data

This section covers how a C++ program actually arranges its data inside the limited RAM of a microcontroller, and how a developer keeps control over that arrangement. It follows the same convention used in the companion report: every technical term is explained in plain language the first time it appears, every idea is introduced with an everyday analogy before the formal definition, and every claim that depends on a specific platform is tied to that platform explicitly rather than stated as a universal rule.

Three questions organise the whole section. First, how does a developer stop using bare numbers to represent states and start using types the compiler can check? That is the enum discussion in Sections 1.1 to 1.3. Second, where exactly does the compiler place each member of a struct in memory, and why does it sometimes leave gaps? That is alignment and padding, in Section 1.4. Third, how does a developer force a struct to match a byte layout defined by someone else, such as a wire protocol, and what does that cost? That is `__attribute__((packed))` and unions, in Sections 1.5 and 1.6.

1.0 Why Data Layout Is a Design Decision, Not a Detail

On a desktop machine with 16 GB of RAM, the exact byte layout of a struct is rarely worth a developer's attention. On an embedded target it is a first-order design concern for two separate reasons, and the two reasons pull in opposite directions.

The first reason is capacity. The classic ESP32 has roughly 520 KB of internal SRAM on the die, and a meaningful share of that is consumed before the application starts by the Wi-Fi and Bluetooth stacks, the FreeRTOS kernel, and its task stacks. A struct that wastes four bytes is irrelevant on its own; the same struct stored in an array of five hundred sensor readings wastes two kilobytes of a budget that was never generous to begin with.

The second reason is interoperability. The moment a struct leaves the chip, whether over UART, SPI, I2C, or a radio, it stops being a private arrangement of the compiler's choosing and becomes a contract with whatever is on the other end. The receiver does not see field names. It sees a sequence of bytes and assumes a specific meaning for each one. If the sender's compiler inserted three invisible bytes of padding that the receiver does not expect, every field after that point is misread, and the failure is silent rather than loud.

IN PLAIN TERMS

Think of a struct as a shipping crate with internal dividers. The compiler chooses the divider positions for its own convenience, so the forklift (the CPU) can pick up each item quickly. That is fine while the crate stays in your warehouse.

The trouble starts when you ship the crate to a partner who has their own packing list. If your compiler quietly added spacers between the items and your partner counted from the packing list, everything they unpack is off by a few centimetres, and nothing about the crate itself announces the mistake.

The rest of Section 1 is essentially about deciding, deliberately and per struct, which of those two situations applies: is this data private to the chip, where the compiler should optimise for speed, or is it a wire format, where the developer must dictate the layout exactly.

1.1 Enums: Giving Names to Integers

An **enum**, short for enumeration, is a user-defined type that attaches meaningful names to a fixed set of integer values. It exists because embedded firmware is full of variables that can only ever hold one value

from a short, closed list: a motor is stopped, running, or faulted; a Wi-Fi link is disconnected, connecting, or connected; a traffic light is red, yellow, or green.

Listing 1.1: the same information, with and without an enum.

```
int motorState = 2;           // What is 2? The reader has to go and find out.

enum MotorState
{
    STOPPED,
    RUNNING,
    ERROR
};

MotorState motorState = RUNNING; // No lookup required.
```

The improvement is not cosmetic. A comparison such as `if (state == 0)` is unverifiable by anyone reading the code later, including its author six weeks on, whereas `if (state == STOPPED)` carries its own documentation. Enums improve readability, maintainability, debugging (a debugger prints the enumerator name rather than a bare integer), and, in the scoped form discussed below, type safety.

1.1.1 How Values Are Assigned

If the developer assigns nothing, the compiler numbers the enumerators from zero and increments by one. If the developer assigns a value part way through the list, numbering continues from that value.

Listing 1.2: automatic and explicit enumerator values.

```
enum Direction { FORWARD, BACKWARD, STOP };
// Equivalent to: FORWARD = 0, BACKWARD = 1, STOP = 2

enum ErrorCode
{
    OK           = 0,
    SENSOR_ERROR = 100,
    MOTOR_ERROR  = 200,
    WIFI_ERROR   = 300
};

enum Numbers { A = 5, B, C, D };
// Equivalent to: A = 5, B = 6, C = 7, D = 8
```

Deliberately spaced values such as the `ErrorCode` list above are a common embedded convention: the gaps leave room to insert new sensor errors later without renumbering, and the value ranges themselves become self-describing (anything in the 200s is a motor problem).

1.1.2 Underlying Type and Size

Every enumeration is stored as an integer of some **underlying type**, and the enumeration has the same size, value representation, and alignment requirement as that underlying type. For a plain C-style enum whose underlying type is not fixed by the developer, the exact choice is implementation-defined: the standard only requires that the type be capable of representing every enumerator value. In practice the ESP32 toolchain gives a plain enum the size of an `int`, which is 4 bytes.

Since C++11 the developer can fix the underlying type explicitly, which is one of the most useful memory optimisations available in embedded code:

```
enum Color : uint8_t { RED, GREEN, BLUE };

sizeof(Color) // 1 byte instead of 4
```

Three bytes saved per variable sounds trivial. In a struct that is stored in an array of two hundred log entries, or transmitted a hundred times a second, it is not.

1.2 The Two Kinds of Enumeration in C++

C++ has two distinct enumeration mechanisms. The original one, inherited from C, is formally called an **unscoped enumeration** and is what everyone means by "C-style enum". The second, introduced in C++11, is the **scoped enumeration**, declared with `enum class` (or the identical `enum struct`) and often described as a strongly typed enum.

ANALOGY

An unscoped enum is like writing every employee's first name on a single shared whiteboard in the office lobby. It is convenient, until two departments both hire someone called Sara and nobody can tell which one a message is for.

A scoped enum is like giving each department its own whiteboard. You now have to write `Engineering::Sara` instead of just `Sara`, which is two extra keystrokes, and in exchange the ambiguity becomes impossible rather than merely unlikely.

1.2.1 The C-style (Unscoped) Enum and Its Two Structural Problems

Problem one is name pollution. The enumerators of an unscoped enum are injected into the enclosing scope, not kept inside the enum. Two enums in the same scope therefore cannot share an enumerator name:

Listing 1.3: unscoped enumerators collide in the enclosing scope.

```
enum Color { RED, GREEN };
enum TrafficLight { RED, YELLOW, GREEN }; // Compiler error: RED and GREEN redeclared
```

This is not a contrived example. Real firmware routinely wants `RUNNING` in a motor enum, `RUNNING` in a system-state enum, and `ERROR` in three different subsystems. With unscoped enums the team ends up prefixing everything by hand (`MOTOR_RUNNING`, `SYS_RUNNING`), which is exactly the manual namespacing that the language feature was supposed to provide.

Problem two is implicit conversion to int. An unscoped enumerator converts silently to an integer wherever an integer is expected, so the compiler accepts a range of statements that are almost certainly mistakes:

Listing 1.4: an unscoped enum offers almost no protection from misuse.

```
enum State { RUNNING, STOPPED };

State state = RUNNING;

int x = state;           // Allowed: silent conversion to int
if (state == 0) { }      // Allowed: comparing a state to a raw number
state = (State)100;      // Allowed: 100 is not any declared state
int total = state + 5;    // Allowed: arithmetic on a state has no meaning
```

1.2.2 The Scoped Enum (enum class)

A scoped enumeration closes both holes. Its enumerators live inside the enum type and must be qualified, and it does not convert implicitly to an integer at all.

Listing 1.5: scoped enums keep their enumerators and refuse silent conversions.

```
enum class State { RUNNING, STOPPED };
```

```

State state = State::RUNNING;    // Qualification is mandatory

int x = State::RUNNING;          // Compiler error
int y = static_cast<int>(State::RUNNING); // Allowed, but explicit and visible

enum class Color { RED };
enum class TrafficLight { RED }; // Perfectly valid: no collision

```

The conversion still exists, it simply has to be requested with `static_cast`. That is the entire design intent: the operation is not forbidden, it is made visible, so a reviewer can see the exact points where a state is being turned into a number and ask whether that is deliberate.

1.2.3 Side-by-Side Comparison

Aspect	C-style enum (unscoped)	enum class (scoped)
Declaration	<code>enum State { ... };</code>	<code>enum class State { ... };</code>
Access to enumerators	Unqualified: <code>RUNNING</code>	Qualified: <code>State::RUNNING</code>
Scope of enumerators	Leaks into the enclosing scope	Contained inside the enum type
Name collisions	Possible between two enums	Impossible between two enums
Implicit conversion to int	Yes, silent	No, <code>static_cast</code> required
Comparison with another enum type	Compiles (both promote to int)	Does not compile
Default underlying type	Implementation-defined, must be able to hold all enumerators (typically <code>int</code> on ESP32)	Fixed as <code>int</code> by default, and may be changed
Custom underlying type	Allowed since C++11: <code>enum E : uint8_t</code>	Allowed: <code>enum class E : uint8_t</code>
Forward declaration	Requires an explicit underlying type	Works without one, since the default is known
Availability	C and all versions of C++	C++11 and later
Recommended for new C++ code	No	Yes

Table 1.1: unscoped enumeration versus scoped enumeration.

1.3 Why enum class Is Safer for State Machines

A state machine is the single place in embedded firmware where the guarantees above stop being stylistic and start being about correctness. A state variable is, by definition, a variable that must hold exactly one value from a closed list, and where holding anything else means the system is in a condition its author never designed for. That is precisely the invariant an unscoped enum fails to protect and a scoped enum protects by construction.

Six specific mechanisms are worth separating out, because they fail in different ways.

1.3.1 Reason One: Several State Machines Can Coexist

A real device runs more than one state machine at once. A motor controller has motor states; the same firmware has connectivity states, calibration states, and safety states. Many of them naturally want the same words.

Listing 1.6: two state machines, same vocabulary, no conflict.

```

// Unscoped: does not compile, and the workaround is manual prefixing

```

```
enum MotorState { IDLE, RUNNING, FAULT };
enum SystemState { IDLE, RUNNING, FAULT };    // Error

// Scoped: compiles, and reads better at the point of use
enum class MotorState : uint8_t { IDLE, RUNNING, FAULT };
enum class SystemState : uint8_t { IDLE, RUNNING, FAULT };

if (motor == MotorState::FAULT && system != SystemState::FAULT) { /* ... */ }
```

1.3.2 Reason Two: A State Cannot Be Compared Against a Raw Number

With an unscoped enum, `if (state == 1)` compiles cleanly. It is also the single most fragile line a state machine can contain, because the meaning of `1` is defined by the position of an enumerator in a list that somebody will eventually reorder. Reordering an enum is normally a harmless refactor. Combined with a hard-coded comparison it silently redirects the logic to a different state, and no test that only exercises the happy path will catch it.

With a scoped enum, that comparison does not compile at all. The refactor stays harmless.

1.3.3 Reason Three: Two Different State Types Cannot Be Compared By Accident

This is the failure mode that is hardest to spot in review, because the line looks completely reasonable:

Listing 1.7: a comparison between unrelated state machines that the compiler accepts.

```
enum MotorState { M_IDLE, M_RUNNING };
enum WifiState { W_IDLE, W_RUNNING };

MotorState motor = M_RUNNING;
WifiState wifi = W_IDLE;

if (motor == wifi) { /* ... */ }    // Compiles. Both sides became ints.
                                   // It is comparing 1 with 0 and calling that logic.
```

With scoped enums the same comparison is a compile error, because neither operand converts to a common integer type. The mistake is caught at the moment it is typed rather than on a factory floor.

1.3.4 Reason Four: Arithmetic on States Becomes Impossible

Unscoped enums permit `state++`, `state + 1`, and `state * 2`. In a state machine, "advance to the next state by adding one" is a genuinely tempting shortcut and a genuinely bad idea: it makes the transition logic depend on declaration order, it walks off the end of the list at the last state, and it produces a value that is not any declared state without any diagnostic. A scoped enum rejects all of it, forcing transitions to be written explicitly, which is also where they belong, because a transition table is the part of the design a reviewer most needs to read.

1.3.5 Reason Five: Predictable Size and Clean Headers

A scoped enum has a fixed underlying type of `int` by default and accepts any integral type instead, so `enum class MotorState : uint8_t` is exactly one byte with a known representation. This matters twice over. In RAM it shrinks state fields inside structs that may be stored in arrays or queues. On the wire it makes the state field a defined width, which is the difference between a protocol that works between two ESP32s and a protocol that also works when one end is later replaced by a 64-bit gateway.

Because the underlying type is known, a scoped enum can also be forward declared in a header without exposing its enumerator list, which keeps compilation dependencies small. An unscoped enum can only be forward declared if the underlying type is stated explicitly.

1.3.6 Reason Six: The Compiler Can Check That Every State Is Handled

When a `switch` operates on an enumeration type and has no `default: label`, GCC's `-Wswitch` warning (enabled by `-Wall`, which the ESP-IDF build system turns on) reports every enumerator that no case handles. Adding a new state to the machine then produces a compile-time list of every `switch` statement that needs updating, which is by far the most valuable safety net a growing state machine can have.

WHY THIS MATTERS

Be precise about this one: `-Wswitch` applies to unscoped enums too. The difference is that it only fires when the `switch` operand still has the enum type, and with an unscoped enum the state has a habit of quietly decaying into an `int` along the way, through an assignment, a struct field, or a function parameter that someone typed as `int`. Once that happens the warning disappears and nobody notices it is gone.

A scoped enum keeps the type intact end to end, so the check keeps working. The rule of thumb: prefer an explicit case per state over a `default: label`, because `default: label` also switches the warning off.

1.3.7 What enum class Does Not Do

One honest limitation, because it is the assumption that catches people out. A scoped enum is not a validator. `static_cast<MotorState>(42)` is still legal, and for an enumeration with a fixed underlying type every value of that underlying type is a valid value of the enumeration, even if no enumerator names it. So a state decoded from a received packet or read from flash can still be a number that no case label handles.

The type system protects the inside of the program. Data crossing a boundary still needs an explicit check:

Listing 1.8: validating an externally supplied state at the boundary.

```
bool decodeMotorState(uint8_t raw, MotorState &out)
{
    switch (static_cast<MotorState>(raw))
    {
        case MotorState::IDLE:
        case MotorState::RUNNING:
        case MotorState::FAULT:
            out = static_cast<MotorState>(raw);
            return true;
        default:
            return false;        // Reject anything the protocol never defined
    }
}
```

1.3.8 Worked Example: A Motor State Machine

Putting the six points together, the recommended form for the motor protection work from the companion report looks like this:

Listing 1.9: a one-byte, type-safe, compiler-checked motor state machine.

```
enum class MotorState : uint8_t
{
    IDLE,
    RUNNING,
    EMERGENCY_STOP,
    FAULT
};

MotorState state = MotorState::IDLE;

switch (state)                // No default label, so -Wswitch guards the list
{
    case MotorState::IDLE:
        motor.stop();
```

```

        break;

    case MotorState::RUNNING:
        motor.forward();
        if (currentSensor.isStalled()) state = MotorState::FAULT;
        break;

    case MotorState::EMERGENCY_STOP:
        motor.stop();
        break;

    case MotorState::FAULT:
        motor.stop();
        relay.open();
        break;
}

```

Add a fifth state, `CALIBRATING`, and the build immediately points at this switch and every other one in the project. That is the property worth paying two extra keystrokes per enumerator for.

1.3.9 Best Practice Summary

- Prefer `enum class` over a plain `enum` in all new C++ code, and especially for anything representing a state, a mode, a command, or an error code.
- Specify a small underlying type such as `uint8_t` when the value list is short and the enum will be stored in arrays or transmitted.
- Use enums for closed lists of discrete conditions, never for values that vary continuously such as speed, temperature, or a sensor reading.
- Avoid `default:` in a switch over a complete state list, so the compiler keeps checking exhaustiveness for you.
- Validate any state that arrives from outside the program (a packet, a file, non-volatile storage) before trusting it.
- Prefer an enum over `#define` constants in every case: macros have no type, ignore scope, and are invisible to the debugger.

1.4 Struct Padding and Memory Alignment

The second half of Section 1 moves from what a value means to where it physically sits. The behaviour described here surprises most people the first time they meet it: a struct containing one `char` and one `int` occupies eight bytes, not five, and the two missing bytes are not a compiler bug.

1.4.1 Four Definitions Everything Else Depends On

Memory address. Every byte of RAM has a numeric address. A variable occupies one or more consecutive addresses.

Size. How many bytes a variable occupies, reported by `sizeof()`. On the ESP32 toolchain: `char` is 1, `short` is 2, `int` is 4, `float` is 4, and `double` is 8.

Alignment. Which addresses a variable is allowed to begin at, reported by `alignof()`. If `alignof(int)` is 4, an `int` may start at address 0, 4, 8, 12, and so on, but not at 1, 2, 3, 5, or 6.

Padding. Unused bytes the compiler inserts so that the next member begins at an address its alignment allows.

IN PLAIN TERMS

`sizeof()` answers "how much room does this take?" and `alignof()` answers "where is it allowed to start?" They are completely different questions, and they frequently have different answers.

A useful mental image: a bookshelf with fixed slot markings every four centimetres. A book might be eight centimetres wide (size), yet the rule is still that its spine has to line up with a four-centimetre marking (alignment). The width and the marking spacing are independent facts.

Listing 1.10: size and alignment vary independently across platforms.

```
sizeof(double)    // 8 on both ESP32 and a desktop PC

alignof(double)    // 4 on ESP32
                  // 8 on a typical 64-bit desktop

// Same type, same size, different alignment rule.
```

1.4.2 Why Alignment Exists At All

A CPU does not read memory one byte at a time. It reads in natural word-sized chunks, four bytes at a time on a 32-bit core. If a four-byte integer starts at address 0, one read covers it entirely. If the same integer starts at address 1, it straddles the boundary between two chunks, and the processor has to perform two reads plus shift-and-combine work to reassemble the value. On some architectures the attempt does not merely cost extra instructions, it raises a hardware fault.

Alignment is therefore a bargain: the compiler spends a few bytes of memory to guarantee that every load and store is a single, fast, always-legal operation.

1.4.3 Padding Between Members

Listing 1.11: three padding bytes inserted so that b starts at a multiple of 4.

```
struct Test
{
    char a;    // 1 byte
    int b;     // 4 bytes
};

// Intuition says 1 + 4 = 5 bytes. The compiler produces:
//
// Address 0   a
// Address 1   padding
// Address 2   padding
// Address 3   padding
// Address 4   b
// Address 5   b
// Address 6   b
// Address 7   b
//
// sizeof(Test) == 8
```

1.4.4 The Alignment of the Struct Itself

A struct inherits the strictest alignment requirement of any of its members. In `struct Test` above, `char` requires 1 and `int` requires 4, so `alignof(Test)` is 4, and every instance of `Test` must itself begin at an address divisible by four.

1.4.5 End Padding, and Why Arrays Cause It

Padding also appears after the final member. The reason is arrays: every element of an array must satisfy the struct's own alignment, so the total size has to be a multiple of that alignment.

Listing 1.12: end padding keeps every array element correctly aligned.

```
struct Example
{
    char a;
    int  b;
    char c;
};

// 0 a
// 1 padding
// 2 padding
// 3 padding
// 4 b
// 5 b
// 6 b
// 7 b
// 8 c
// 9 padding <-- end padding
// 10 padding
// 11 padding
//
// sizeof(Example) == 12

Example arr[2];
// Without the end padding the second element would begin at address 9,
// and its int member b would land on address 13, which is not a multiple of 4.
```

1.4.6 How Member Order Changes RAM Usage

This is the practical payoff of the whole subsection, and it is free. Rearranging the same three members changes the size of the struct by a third.

	struct Bad	struct Good
Declaration order	char a; int b; char c;	int b; char a; char c;
Layout	0 a 1 to 3 padding 4 to 7 b 8 c 9 to 11 padding	0 to 3 b 4 a 5 c 6 to 7 padding
Size	12 bytes	8 bytes
Array of 100	1,200 bytes	800 bytes
Saving	Baseline	400 bytes, one third less
Runtime cost	None	None
Change to the data	None	None

Table 1.2: identical data, identical performance, one third less RAM.

WHY THIS MATTERS

The rule that produces this result is simple enough to apply without thinking: **declare members from the largest alignment requirement down to the smallest.** Doubles and 64-bit integers first, then 32-bit types, then 16-bit, then bytes and booleans last.

It is the cheapest optimisation in embedded programming. It costs one minute of attention per struct, changes nothing about the behaviour of the code, and it is the first thing worth trying when a build starts running short on heap.

1.4.7 The ABI Decides the Rules, Not the CPU

A common assumption is that alignment is a property of the processor. It is more accurate to say it is a property of the **ABI**, the Application Binary Interface, which is the set of binary conventions the toolchain follows for the target platform.

An **API** (Application Programming Interface) tells the programmer how to write source code: function names, parameters, return types. An **ABI** tells the compiler how to organise the compiled machine code: structure alignment, padding rules, function calling conventions, register usage, stack layout, and parameter passing. An API is a source-code contract; an ABI is a machine-code contract.

The toolchain, meaning the compiler together with its target platform, chooses the ABI. For the classic ESP32 that toolchain is `xtensa-esp32-elf-g++`; for the RISC-V parts such as the ESP32-C3 it is `riscv32-esp-elf-g++`. Neither is configured by hand in normal work.

Listing 1.13: identical source, two ABIs, two different memory layouts.

```
struct Test
{
    char    a;
    double b;
};

// ESP32 (alignof(double) == 4):      desktop x86-64 (alignof(double) == 8):
//  0  a                               0  a
//  1  padding                         1 to 7  padding
//  2  padding                         8 to 15 b
//  3  padding
//  4 to 11  b                         sizeof == 16
//
// sizeof == 12
```

The ESP32 is a 32-bit CPU, which is often misread as meaning it cannot handle 64-bit data. It can. A 32-bit CPU means its general-purpose registers are 32 bits wide, most instructions naturally operate on 32-bit values, and addresses are 32 bits. A `double` is still 8 bytes on the ESP32; the CPU simply loads it in two halves and combines them. Data size and CPU word size are separate concepts, and the ABI, not the register width, is what decides the alignment.

WHY THIS MATTERS

The practical consequence is a rule rather than a fact to memorise: **do not assume a layout that worked on one target carries over to another**. The same source file compiled for the classic ESP32 and for an ESP32-C3 is compiled by two different toolchains, and a wire format that relied on the incidental layout of one of them is a bug waiting for a hardware revision.

Verify instead of assuming, using the tools in the next subsection.

1.4.8 Tools for Inspecting Layout

Listing 1.14: measuring layout, and freezing it with `static_assert`.

```
#include <cstdint>

struct Test
{
```

```

char    a;
int     b;
double  c;
};

Serial.println(sizeof(Test));           // Total size in bytes
Serial.println(alignedof(Test));        // Required starting alignment
Serial.println(offsetof(Test, a));      // Where each member actually begins
Serial.println(offsetof(Test, b));
Serial.println(offsetof(Test, c));

// Better still, make the requirement a build-time contract:
static_assert(sizeof(Test) == 16, "Test layout changed, wire format is broken");

```

The `static_assert` on the last line deserves emphasis. Any struct that defines a wire format should carry one. It converts a silent, remote, intermittent field failure into a compile error on the developer's own machine the moment somebody adds a member or changes a target.

1.5 What `__attribute__((packed))` Does

`__attribute__((packed))` is a GCC and Clang extension that instructs the compiler to insert no padding between members and to give the struct the smallest possible alignment. It is the tool for the second scenario from Section 1.0: a struct whose layout is dictated by something outside the program.

Listing 1.15: the same struct with and without packing.

```

struct Example
{
    char a;
    int  b;
};
// sizeof == 8    (three padding bytes)

struct __attribute__((packed)) PackedExample
{
    char a;
    int  b;
};
// sizeof == 5    (no padding at all)
//  0  a
//  1 to 4  b

```

1.5.1 What Packing Does Not Do

Four misconceptions are worth stating explicitly, because each one produces a different class of bug.

- **It does not shrink types.** An `int` is still 4 bytes inside a packed struct. Packing removes the gaps between members, nothing more.
- **It does not change byte order.** ESP32 chips, both Xtensa and RISC-V, are little-endian. Many network protocols define fields in big-endian (network byte order). A packed struct sent to a big-endian peer will still be misread, one field at a time.
- **It is not portable syntax.** `__attribute__((packed))` is a GCC and Clang extension. MSVC uses `#pragma pack`. There is no standard C++ spelling, so a header shared with a Windows-built tool needs conditional compilation.

- **It does not make the format self-describing.** Both ends still have to agree on the struct definition by hand. Packing enforces the layout; it does not communicate it.

1.5.2 The Cost: Unaligned Members

Packing works by allowing members to start at addresses their type would normally forbid. In `PackedExample` above, `b` begins at address 1. Depending on the processor, reading it can require more instructions, multiple memory accesses, extra load and store operations, or, on the strictest architectures, it can fault outright.

A second, subtler consequence: the alignment of the packed struct itself drops, commonly to 1 on GCC and Clang. Any pointer or reference taken to a member of a packed struct is therefore potentially unaligned, and GCC warns about it under `-Waddress-of-packed-member`. Passing `&packet.timestamp` to a function that expects a properly aligned `uint32_t*` is the usual way this bites.

1.5.3 The ESP32-Specific Picture

FACT CHECK

The common advice that "packed structs are slower" needs qualifying on this platform. Espressif's own documentation states that **all ESP chips, on both the Xtensa and RISC-V architectures, allow unaligned memory access and incur no extra penalty**, and that the GCC warning about accessing an unaligned member of a packed struct can be ignored in most cases.

That leniency is not unconditional. Unaligned access to **external PSRAM** has been reported to raise a `LoadStoreAlignment` panic on ESP32 devices, so a packed struct that lives in internal SRAM and a packed struct allocated in PSRAM are not equally safe.

Practical rule for ESP32 work: packing a protocol struct that lives in internal RAM is fine and is what Espressif's own example code does. Packing a large buffer that you intend to allocate in PSRAM deserves testing before you trust it.

1.5.4 When to Use Packed, and When Not To

Use packed when	Do not use packed when
The layout must match an external wire format exactly (UART, SPI, I2C, CAN, Ethernet, ESP-NOW payloads)	The struct is ordinary internal application data that never leaves the chip
Parsing a binary file format such as <code>.bin</code> , <code>.bmp</code> , or <code>.wav</code> , where every byte offset is defined by the specification	You are only trying to save RAM, and reordering the members would have achieved the same result for free
Mapping a hardware register block where padding would shift every subsequent address	The struct will be allocated in PSRAM on an ESP32 without being tested first
Building network packets in which every byte has a predefined meaning	Members will have pointers or references taken to them and passed elsewhere

Table 1.3: deciding whether a struct should be packed.

1.5.5 Worked Example: An ESP-NOW Telemetry Packet

This example connects directly to Section 2. ESP-NOW hands the application a raw byte buffer with no serialisation layer of its own, so what the sender lays out in memory is literally what arrives at the receiver.

Listing 1.16: a packed telemetry packet with a build-time size contract.

```
struct Packet
{
    uint8_t id;
    uint16_t speed;
    uint32_t timestamp;
};
// Normal layout: id, 1 padding byte, speed, 2 padding bytes, timestamp = 8 bytes
```

```

struct __attribute__((packed)) Packet
{
    uint8_t id;           // offset 0
    uint16_t speed;        // offset 1
    uint32_t timestamp;    // offset 3
};
static_assert(sizeof(Packet) == 7, "ESP-NOW packet layout changed");

// 7 bytes on the wire, every byte where the receiver expects it.

```

Espressif's own `ESP_NOW_Network` example in the `arduino-esp32` repository uses `__attribute__((packed))` on its message struct for exactly this reason, with a comment noting that it ensures the data is contiguous and without gaps.

1.5.6 The More Portable Alternative: Explicit Serialisation

Packing is convenient but it still leaves the byte order and the compiler extension as unstated assumptions. When a protocol has to survive a change of compiler, of architecture, or of the device on the other end, the durable approach is to write the bytes out explicitly:

Listing 1.17: serialisation that makes every assumption visible.

```

size_t encodePacket(const Packet &in, uint8_t *out)
{
    size_t i = 0;
    out[i++] = in.id;
    out[i++] = (uint8_t)(in.speed & 0xFF);           // little-endian, stated
    out[i++] = (uint8_t)(in.speed >> 8);
    out[i++] = (uint8_t)(in.timestamp & 0xFF);
    out[i++] = (uint8_t)(in.timestamp >> 8);
    out[i++] = (uint8_t)(in.timestamp >> 16);
    out[i++] = (uint8_t)(in.timestamp >> 24);
    return i;    // Always 7, on every compiler and every architecture
}

```

It is more code. It is also the version that still works in three years when one node in the network is a Linux gateway and the packed-struct version has quietly started reading garbage.

1.6 Unpacking Data: Unions and Byte Views

A **union** is a user-defined type in which every member starts at the same address and shares the same storage. Writing to one member overwrites the others. Where a struct gives each member its own space, a union gives all members one space and lets the program choose which interpretation to apply.

IN PLAIN TERMS

A struct is a chest of drawers: every item has its own drawer, and they coexist.

A union is a single drawer with several labels on the front. Only one of the labels is telling the truth at any moment, and the program has to remember which.

Listing 1.18: union size and alignment follow the largest member.

```

union Data
{
    char c;    // 1 byte
    int i;     // 4 bytes
    double d;  // 8 bytes
};

```

```
sizeof(Data)    // 8, the size of the largest member, not 13
alignof(Data)   // The largest alignment required by any member
```

1.6.1 The Three Reasons Unions Appear in Embedded Code

Saving RAM when only one of several values is meaningful at a time. A struct holding a temperature, a humidity, and a status allocates room for all three; a union allocates room for one.

Viewing the same bytes two ways, which is the reason the topic sits under "unpacking data". Sending a float over UART, SPI, or I2C requires its four raw bytes, and a union offers them without any copying:

Listing 1.19: a byte view of a float, with no conversion step.

```
union Number
{
    float    value;
    uint8_t  bytes[4];
};

Number num;
num.value = 3.14f;

for (int i = 0; i < 4; i++)
{
    Serial.write(num.bytes[i]);    // The float, one byte at a time
}
```

Mapping hardware registers, where a 32-bit register needs to be addressable both as a whole word and as individual control bits:

Listing 1.20: register access through a union of a word and a bitfield struct.

```
union Register
{
    uint32_t value;
    struct
    {
        uint32_t enable    : 1;
        uint32_t ready     : 1;
        uint32_t error     : 1;
        uint32_t reserved  : 29;
    };
};

reg.value = 0x05;    // Write the whole register
reg.enable = 1;      // Or a single bit of the same register
```

1.6.2 Two Caveats Worth Knowing Before Relying On This

IMPORTANT RULE

Type punning. Writing one member of a union and reading a different one is well defined in C but is not, strictly, well defined in standard C++. GCC (including the compiler used for the ESP32) supports it as a documented extension and it is standard practice in low-level code, so it works in this context. Portable C++ should prefer `std::memcpy` into a byte array, which compilers optimise away to nothing, or `std::bit_cast` in C++20.

Bitfields are not a wire format. The order in which a compiler allocates bitfields within a word, and how it straddles storage units, is implementation defined. Bitfield unions are excellent for registers on a known toolchain and a poor choice for a protocol that has to be parsed by a different compiler on the other end.

1.6.3 Struct versus Union

Feature	Struct	Union
Memory	Separate storage for each member	One shared storage for all members
Size	Sum of members plus padding	Size of the largest member, rounded up for alignment
Alignment	Largest alignment among members	Largest alignment among members
Members valid at once	All of them	One at a time
Writing to one member	Leaves the others intact	Overwrites the others
Typical use	Store related data together	Store one of several possible interpretations
Embedded use case	Sensor records, packets, configuration	Byte views, register maps, memory saving

Table 1.4: struct and union compared.

1.7 Key Terms at a Glance

Term	Plain-language definition
<code>sizeof()</code>	How many bytes an object occupies
<code>alignof()</code>	Which addresses an object is allowed to begin at
<code>offsetof()</code>	Where a given member actually starts inside its struct
Padding	Unused bytes the compiler inserts to satisfy alignment
End padding	Padding after the last member, so array elements stay aligned
ABI	Application Binary Interface: the binary rulebook (alignment, padding, calling conventions, register usage) that a toolchain follows for a target
API	Application Programming Interface: the source-level contract of names, parameters, and return types
Unscoped enum	C-style enum: enumerators leak into the enclosing scope and convert implicitly to int
Scoped enum	<code>enum class</code> : enumerators stay inside the type and require an explicit cast to become an integer
Underlying type	The integer type an enumeration is actually stored as, which fixes its size and alignment
<code>__attribute__((packed))</code>	GCC and Clang instruction to remove all padding from a struct
Type punning	Reading the same bytes through a different type than the one used to write them
Little-endian	Byte order in which the least significant byte is stored first; used by all ESP32 variants

Table 1.5: quick-reference glossary for Section 1.

1.8 Section 1 Recap

- An enum gives names to integers; a scoped enum (`enum class`) additionally keeps those names inside the type and refuses to convert them to integers silently.
- For state machines, `enum class` prevents name collisions between machines, comparisons against raw numbers, comparisons between unrelated state types, and arithmetic on states, while keeping `-wswitch` exhaustiveness checking effective.

- A scoped enum is not a validator: any value of the underlying type is representable, so states arriving from outside the program still need an explicit check.
- Alignment determines where a member may begin; padding is what the compiler inserts to make that happen; the struct itself inherits the strictest alignment of its members.
- Ordering members from the largest alignment requirement to the smallest reduces padding, and therefore RAM, at no cost to performance or behaviour.
- The ABI, chosen by the toolchain, decides alignment rules, so the same source can produce different layouts on the ESP32 and on a desktop, or between Xtensa and RISC-V ESP variants.
- `__attribute__((packed))` removes padding so a struct matches an external byte layout exactly. It does not shrink types, change endianness, or port across compilers.
- On ESP chips, Espressif documents unaligned access as carrying no extra penalty in internal memory, but unaligned access to external PSRAM has been observed to fault, so packed data in PSRAM warrants testing.
- A union lets one block of storage be interpreted several ways, which is how raw bytes are extracted from a float or mapped onto a hardware register. Use `memcpy` where portability matters, and never use bitfields as a cross-compiler wire format.
- Any struct that defines a wire format should carry a `static_assert` on its size, so that a layout change fails the build instead of the product.

Section 2: IoT Communication Protocols (HTTP, MQTT, and ESP-NOW)

Section 1 dealt with how data is arranged inside one chip. This section deals with how that data gets to another one. The companion report covered the four wired serial protocols (UART, I2C, SPI, RS-485), which move bytes between chips on the same board or along a cable inside the same building. The three protocols here move bytes between whole devices, over a radio.

The three are not variations on a theme. HTTP is a request and response protocol built for the web and reused for embedded work. MQTT is a publish and subscribe protocol designed from the start for machine-to-machine messaging. ESP-NOW is a connectionless Espressif protocol that skips the IP network entirely. Choosing between them is an architectural decision that shapes the power budget, the latency, and the failure behaviour of the whole product, so this section analyses each against the same criteria and then places them side by side.

2.0 Where These Protocols Sit

A communication protocol defines how devices talk to each other: how messages are formatted, who sends, who receives, whether delivery is guaranteed, and how devices find one another. It is the language two devices agree on in advance. Where each protocol sits in the stack explains most of its behaviour.

Figure 2.1: HTTP and MQTT ride the full IP stack; ESP-NOW attaches directly to the Wi-Fi MAC layer.

Application	HTTP	MQTT	ESP-NOW
Transport	TCP	TCP	(no TCP, no UDP)
Network	IP	IP	(no IP address at all)
Data link	802.11 MAC	802.11 MAC	802.11 vendor-specific action frame
Physical	2.4 GHz radio waves		

That single diagram predicts most of the comparison table later in this section. HTTP and MQTT inherit everything TCP/IP provides, including reliable ordered delivery, routing across the internet, and universal interoperability, and they also inherit its costs: association with an access point, DHCP, handshakes, and per-message header overhead. ESP-NOW gives all of that up and receives in exchange the two things TCP/IP cannot offer a battery-powered sensor: near-zero setup time and no infrastructure.

CONNECTING BACK TO SECTION 1

HTTP and MQTT usually carry text or JSON, which is self-describing: a field is found by its name, so struct padding is irrelevant.

ESP-NOW hands the application a raw byte buffer with no serialisation layer of its own. What the sender laid out in memory is exactly what the receiver has to parse. This is the case where the packed structs and the `static_assert` from Section 1.5 stop being good practice and start being load-bearing.

2.1 HTTP (HyperText Transfer Protocol)

HTTP was created to move web pages between browsers and web servers. It has since become one of the most common protocols in embedded systems, largely because every phone, laptop, and cloud service already speaks it.

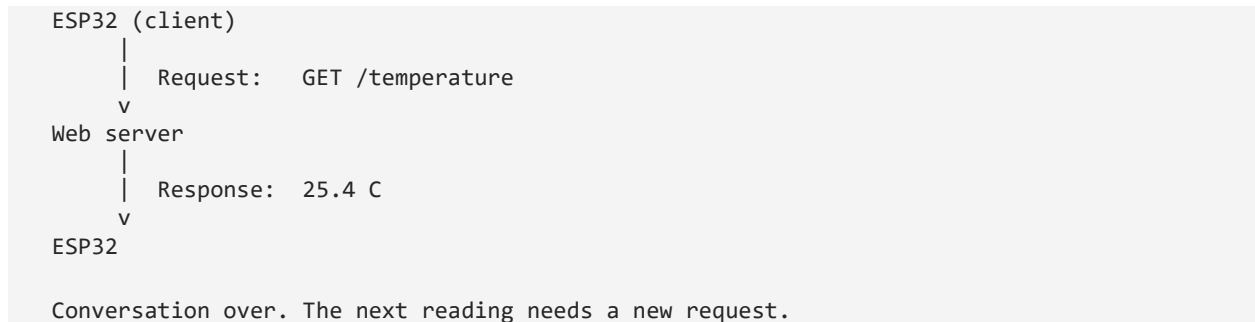
IN PLAIN TERMS

HTTP is a phone call. You dial, you ask one question, you get one answer, and you hang up. If you want to know something else a minute later, you dial again from the beginning.

The person on the other end never calls you. If you want to know the instant something changes, your only option is to keep calling and asking.

2.1.1 How It Works

Figure 2.2: the HTTP request and response cycle.



HTTP follows a strict client and server model. The client always starts the exchange; the server only ever answers. The protocol is **stateless**, meaning the server keeps no memory of previous requests, so every request must carry all the context it needs. That is a virtue for scalability on the web and a cost for a sensor node, because that context is retransmitted every single time.

2.1.2 Overhead, and the Polling Problem

Each HTTP request carries a block of human-readable text headers before the payload:

Listing 2.1: a typical header block, repeated on every request.

```
GET /temperature HTTP/1.1
Host: example.com
User-Agent: ESP32HTTPClient
Content-Type: application/json
Authorization: Bearer eyJhbGciOi...
Connection: keep-alive
```

The actual payload might be the two characters `25`. The headers around it are commonly a few hundred bytes. HTTP keep-alive lets several requests reuse one TCP connection, which removes the repeated handshake, but each request still carries its own header block, so the per-message overhead never goes away.

The deeper cost is architectural. Because the server cannot push, a device that needs to react to a remote command has to poll: ask every second whether anything has changed, and receive "no" almost every time. Polling burns radio time, burns battery, and still leaves an average latency of half the polling interval. WebSockets and Server-Sent Events exist precisely to work around this, and both are additional protocols layered on top rather than something HTTP provides.

2.1.3 Where HTTP Is the Right Answer on an ESP32

- Hosting a local web dashboard that a phone or laptop opens in a browser.
- A first-time configuration portal, where the device runs a SoftAP and serves a Wi-Fi setup page.
- Exposing a REST API that other software already knows how to call.

- Over-the-air firmware updates, which are one large transfer rather than many small ones.
- Occasional posts to a cloud endpoint that only accepts HTTP.

2.1.4 Advantages and Disadvantages

Advantages of HTTP	Disadvantages of HTTP
Universally supported, by every browser, language, and cloud platform	Request and response only: the server cannot push, so devices must poll
Easy to test and debug with a browser, curl, or Postman	Large protocol overhead relative to a small sensor reading
Works naturally with REST APIs and existing web tooling	Inefficient for frequent small updates
Excellent for dashboards, configuration pages, and file serving	No built-in publish and subscribe, no QoS, no retained messages, no message queuing
No broker or extra infrastructure to run	Higher power consumption for continuous telemetry
Human-readable, which makes field diagnosis simple	TLS for HTTPS costs a significant slice of RAM on a constrained device

Table 2.1: HTTP strengths and weaknesses in an embedded context.

2.2 MQTT (Message Queuing Telemetry Transport)

MQTT is an OASIS standard application-layer protocol designed specifically for machine-to-machine communication, and it has become one of the most widely used protocols in IoT. It runs over TCP/IP like HTTP, but the resemblance ends there. Instead of clients talking directly to each other, every client connects to a central **broker**, and messages are routed by topic.

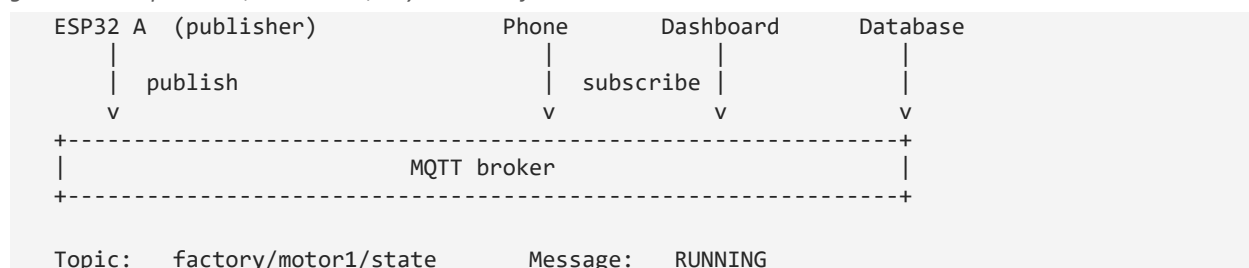
IN PLAIN TERMS

MQTT is a subscription to a news channel. You tell the broker once which topics interest you, and from then on anything published to those topics arrives on its own. You never ask.

The publisher does not know who is listening, and the subscribers do not know who published. That decoupling is the whole design, and it is why adding a tenth sensor to an MQTT network requires no change to the other nine.

2.2.1 Architecture and Topics

Figure 2.3: one publisher, one broker, any number of subscribers.



Instead of addresses, MQTT uses hierarchical **topics** such as `factory/motor1/state`, `factory/temperature`, or `home/livingroom/light`. Subscribers can use wildcards: `+` matches exactly one level, so `factory/+/state` catches the state of every machine, while `#` matches everything below a point, so `factory/#` catches the entire site. No device ever needs to know another device's IP address.

2.2.2 Quality of Service

TCP already guarantees ordered, error-checked, retransmitted delivery between a client and the broker. MQTT adds a second reliability layer on top, called **Quality of Service**, which governs delivery of the application message itself and therefore survives a client dropping out mid-conversation.

Level	Name	Behaviour	Cost	Typical use
QoS 0	At most once	Fire and forget. The sender does not wait for confirmation.	Lowest	Frequent telemetry where the next reading is a second away
QoS 1	At least once	The receiver acknowledges. Duplicates are possible.	Medium	Commands and alerts that must not be lost
QoS 2	Exactly once	Four-part handshake. No loss and no duplicates.	Highest	Billing, counters, and anything a duplicate would corrupt

Table 2.2: MQTT Quality of Service levels.

QoS 1 is the workhorse. QoS 2 is genuinely expensive in round trips and is only worth it where a duplicated message would cause real damage.

2.2.3 The Features That Have No HTTP Equivalent

Retained messages. The broker stores the most recent message on a topic. A device that subscribes later receives that last known value immediately rather than waiting for the next update. A dashboard opened at three in the morning shows the motor state instantly instead of a blank field.

Last Will and Testament. A client registers a message with the broker at connection time. If that client disconnects ungracefully, through a power cut or a radio failure, the broker publishes the message on its behalf. Every other device learns that a node has died within seconds, without anyone polling for it. For safety-adjacent systems this is close to indispensable.

Persistent sessions and queued messages. With a persistent session, the broker holds messages for a subscriber that has temporarily dropped off and delivers them on reconnection, which matters over unreliable cellular or congested Wi-Fi.

Minimal header. The MQTT fixed header starts at two bytes. Compared with a few hundred bytes of HTTP headers, the difference for a device publishing a reading every second is the difference between a battery lasting weeks and lasting months.

Keep-alive. The client and broker agree on a keep-alive interval, and a short ping keeps the connection recognised as live, so the broker detects a dead client without the application implementing its own heartbeat.

2.2.4 What MQTT Costs

A broker has to exist and has to be run by someone. That can be Eclipse Mosquitto on a Raspberry Pi on the same LAN, or a managed cloud service such as HiveMQ or EMQX. It is one more component to deploy, secure, and keep alive, and while enterprise brokers cluster to millions of connections, a single self-hosted broker is a single point of dependency for the whole fleet.

Two smaller points. Security is TLS on port 8883, plus username and password or client certificates, and TLS carries the same RAM cost on an ESP32 as it does for HTTPS. And a browser cannot speak plain MQTT: web dashboards use MQTT over WebSockets, which the broker must be configured to accept.

2.2.5 Where MQTT Is the Right Answer on an ESP32

- Several devices exchanging data, where point-to-point wiring of every pair would not scale.
- Continuous telemetry: a node publishing readings every second, for hours.

- Remote commands sent to a device that sits behind a home or factory router with no public address.
- Cloud and smart-home integration, including Home Assistant, which is MQTT-native.
- Any system where knowing that a node went offline matters, thanks to Last Will and Testament.

2.2.6 Advantages and Disadvantages

Advantages of MQTT	Disadvantages of MQTT
Designed for IoT from the beginning, rather than adapted to it	Requires a broker: an extra component to deploy, secure, and monitor
Very low overhead, with a fixed header as small as two bytes	Slightly more complex to set up and reason about than HTTP
Publish and subscribe decouples senders from receivers entirely	A self-hosted broker is a single point of dependency for the fleet
Built-in QoS levels for tuning reliability against cost	Browsers cannot use plain MQTT and need MQTT over WebSockets
Retained messages give new subscribers the current value at once	Binary payloads are less immediately readable when debugging by hand
Last Will and Testament reports unexpected disconnections automatically	Still depends on Wi-Fi association and a routed network being available
One long-lived connection carries many messages in both directions	Topic naming needs a convention agreed early, or it becomes unmanageable

Table 2.3: MQTT strengths and weaknesses.

2.3 ESP-NOW (Espressif Connectionless Protocol)

ESP-NOW is a connectionless Wi-Fi communication protocol defined by Espressif. Application data is encapsulated in an IEEE 802.11 vendor-specific action frame and transmitted from one Wi-Fi device to another without any connection being established. There is no router, no access point, no internet, no IP address, no TCP, and no UDP.

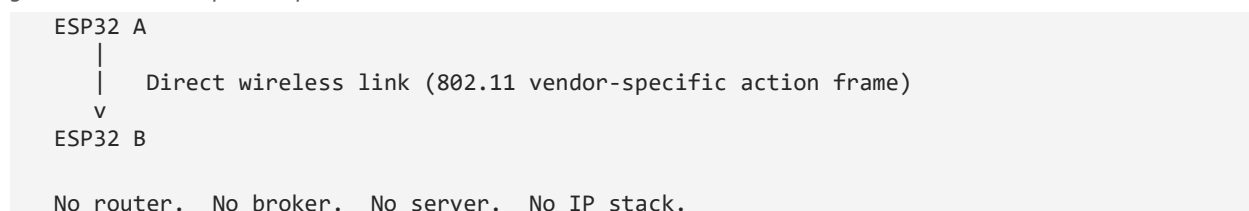
IN PLAIN TERMS

ESP-NOW is a pair of walkie-talkies. Two devices agree on a channel in advance and talk directly. There is no phone network in between, nothing to sign in to, and nothing to wait for.

The trade is exactly what you would expect from walkie-talkies: it is instant and it works when the infrastructure is down, but it only reaches as far as the radio does, only talks to the same brand of handset, and each transmission is short.

2.3.1 How It Works, and the Numbers That Constrain It

Figure 2.4: ESP-NOW peer-to-peer communication.



The specifics below come from Espressif's ESP-IDF programming guide and are worth knowing precisely, because most of them are hard limits rather than tuning parameters.

Property	Value
Maximum payload, ESP-NOW v1.0	250 bytes (ESP_NOW_MAX_DATA_LEN), limited by the single-byte length field of the 802.11 vendor-specific element
Maximum payload, ESP-NOW v2.0	1,470 bytes (ESP_NOW_MAX_DATA_LEN_V2) on devices and IDF versions that support it
Mixed-version networks	A v1.0 device receiving a longer v2.0 packet truncates it to 250 bytes or drops it, so keep to 250 bytes wherever versions are mixed
Paired peers	Up to 20 at the protocol layer
Encrypted peers	No more than 17, with a default of 7 in current IDF versions (older versions: 16, default 6)
Broadcast receivers	Not limited by the peer count, since broadcast frames are not paired
Encryption	CCMP as defined in IEEE 802.11-2012, using a 16-byte Primary Master Key and a 16-byte Local Master Key per peer, with AES-128
Broadcast encryption	Not supported: multicast and broadcast frames cannot be encrypted
Default bit rate	1 Mbps
Channel	Peers must be on the same channel (0 to 14, where 0 means the current channel)

Table 2.4: ESP-NOW protocol limits, from the ESP-IDF programming guide.

2.3.2 Two Points the Common Summaries Get Wrong

FACT CHECK

"ESP-NOW has no acknowledgements." Not quite. For unicast, the send callback registered with `esp_now_register_send_cb()` reports whether the frame was acknowledged at the MAC layer by the peer. That is a real delivery signal, and it is more than UDP gives you. What it is not is an application-level acknowledgement: it confirms the peer's radio received the frame, not that the peer's firmware processed it, and broadcast frames carry no acknowledgement at all. Anything that must be confirmed end to end still needs an application-level reply.

"ESP-NOW and Wi-Fi mix freely." This is the single most common practical failure. ESP-NOW peers have to be on the same Wi-Fi channel, and if the device is also connected to an access point, the access point dictates that channel. A gateway that joins a router on channel 6 will silently stop hearing sensors that are transmitting on channel 1. The usual fixes are to pin the access point to a fixed channel, to have the sensors follow the gateway's channel, or to give the gateway a second ESP32 dedicated to the ESP-NOW side.

2.3.3 Where ESP-NOW Is the Right Answer

- Emergency stop and interlock signalling between two boards, where latency is a safety property.
- Remote controls and handheld pendants.
- Battery sensor nodes that wake, transmit a few dozen bytes, and sleep again, where association time would dominate the energy budget.
- Robotics and motor control links between boards on the same machine.
- Any deployment where there is no router, or where depending on one is an unacceptable failure mode.

2.3.4 Advantages and Disadvantages

Advantages of ESP-NOW	Disadvantages of ESP-NOW
No router, access point, broker, or internet required	Espressif ecosystem only: ESP32 and ESP8266 families, not a general standard
Extremely low latency, with no association, DHCP, or TCP handshake	Small payload: 250 bytes per packet in v1.0, 1,470 in v2.0
Very low power, which suits deep-sleep sensor nodes	Peer limits: 20 paired devices, and no more than 17 of them encrypted
Works during an internet or router outage	Range is limited to direct radio reach, with no routing or forwarding
Built-in CCMP encryption with per-peer keys	Broadcast traffic cannot be encrypted
Unicast frames report MAC-layer delivery through the send callback	All peers must share one Wi-Fi channel, which conflicts with joining an access point
Minimal code and no network stack to configure	No topics, no retained state, no queuing, no broker-side history

Table 2.5: ESP-NOW strengths and weaknesses.

2.4 Master Comparison Table

The table below places all three protocols against the criteria that actually drive the choice on a real project.

Attribute	HTTP	MQTT	ESP-NOW
Full name	HyperText Transfer Protocol	Message Queuing Telemetry Transport	Espressif connectionless peer-to-peer protocol
Originally designed for	Web documents	Machine-to-machine IoT messaging	Direct device-to-device links
Layer	Application, over TCP/IP	Application, over TCP/IP	Directly on the 802.11 MAC layer
Transport	TCP	TCP	Vendor-specific action frames, no TCP or UDP
Communication model	Client and server	Publish, broker, subscribe	Peer to peer
Direction	Client initiates every exchange	Bidirectional once connected	Direct messaging either way
Broker or server needed	A server (which the ESP32 can host itself)	Yes, a broker	No
Router or access point needed	Usually yes	Usually yes	No
Internet needed	No for a local server	No for a local broker, yes for a cloud broker	No
IP address needed	Yes	Yes	No, peers are addressed by MAC address
Addressing	URL and IP	Topic hierarchy with wildcards	6-byte MAC address, or broadcast
Header overhead	Hundreds of bytes of text headers per request	Fixed header from two bytes	A small binary frame header
Payload format	Mostly text, binary needs encoding such as Base64	Binary, anything goes	Raw bytes, defined entirely by the application
Maximum payload	Effectively unlimited	Large, broker-dependent	250 bytes (v1.0) or 1,470 bytes (v2.0)
Delivery guarantees	TCP reliability per request	TCP plus QoS 0, 1, or 2	MAC-layer acknowledgement on unicast, none on broadcast
Security	TLS (HTTPS)	TLS on 8883, credentials or certificates	CCMP with AES-128, PMK and per-peer LMK, unicast only
Latency	Medium	Low	Very low
Power consumption	Medium to high	Low	Very low
Range	Wi-Fi coverage, plus the internet beyond it	Wi-Fi coverage, plus the internet beyond it	Direct radio range only
Scalability	Good, with server-side effort	Excellent, thousands of clients per broker	Limited: 20 paired peers
Interoperability	Universal	Broad, across many vendors and platforms	Espressif chips only
Reachable from a browser	Yes, natively	Only over WebSockets	No
Best for	Dashboards, configuration portals, REST APIs, OTA	Telemetry, cloud, sensor fleets, remote commands	Real-time local device-to-device control

Table 2.6: HTTP, MQTT, and ESP-NOW compared across the criteria that drive protocol choice.

2.5 Reading the Differences: Overhead, Latency, and Power

The qualitative labels in the table above (low, medium, very low) are worth unpacking, because they come from mechanisms rather than benchmarks.

2.5.1 HTTP versus MQTT

HiveMQ compared the two protocols using repeated message exchanges. The results are intuitive once the mechanism is clear: HTTP has lower connection setup overhead for a **single** request, because there is no session to establish, no subscription to register, and no keep-alive to maintain. MQTT becomes much more efficient as soon as many messages are exchanged over the same connection, and it also achieved significantly lower average response times in repeated messaging scenarios.

The crossover point is low. A device that reports once an hour is arguably fine on HTTP. A device that reports every second crosses into MQTT territory almost immediately, because it is paying the header cost 3,600 times where MQTT pays a connection cost once.

2.5.2 Why ESP-NOW Latency Is In a Different Category

The gap between ESP-NOW and the other two is not incremental. Before a Wi-Fi device can send its first HTTP or MQTT byte it must scan for the access point, associate, complete the authentication handshake, obtain an IP address by DHCP, open a TCP connection, and, if TLS is in use, complete a TLS handshake as well. That sequence commonly takes hundreds of milliseconds and can take several seconds on a busy network. ESP-NOW does none of it: the radio starts, and the frame goes out.

WHY THIS MATTERS

For a mains-powered dashboard, a two-second connection is invisible. For two devices in the same room that need to agree on stopping a motor, it is the entire problem.

For a battery node, it is the whole energy budget. Energy is roughly current multiplied by time awake, and for a node that sends a handful of bytes, the time awake is dominated almost completely by network setup rather than by transmission. Removing association and handshakes is what turns weeks of battery life into months.

2.6 Which One Should I Use?

2.6.1 Choose HTTP When

- A human needs to look at something in a browser.
- The device needs a configuration page or a first-time setup portal.
- An existing REST API or cloud service is the counterpart and it only speaks HTTP.
- The exchange is occasional and one-off rather than a continuous stream.
- Firmware is being served over the air.

Example: an ESP32 hosts a web page showing live temperature and humidity, which anyone on the local network can open.

2.6.2 Choose MQTT When

- More than two devices need to exchange data, or the number of devices will grow.
- Telemetry is continuous and small.
- Commands arrive from outside the local network.

- Cloud services, databases, or a home automation platform are consuming the data.
- It matters that other parts of the system learn quickly when a node disappears.

Example: ten ESP32 sensors publish to one broker, and a phone, a dashboard, and a database all subscribe without any of them knowing the sensors exist individually.

2.6.3 Choose ESP-NOW When

- Speed is a functional requirement rather than a nicety.
- There is no router available, or depending on one is unacceptable.
- Communication is local, between devices in radio range.
- The nodes run on batteries and spend most of their life asleep.
- Both ends are Espressif chips and will stay that way.

Example: an emergency stop signal between two ESP32 boards on the same machine, which must keep working when the site Wi-Fi does not.

2.6.4 A Short Decision Path

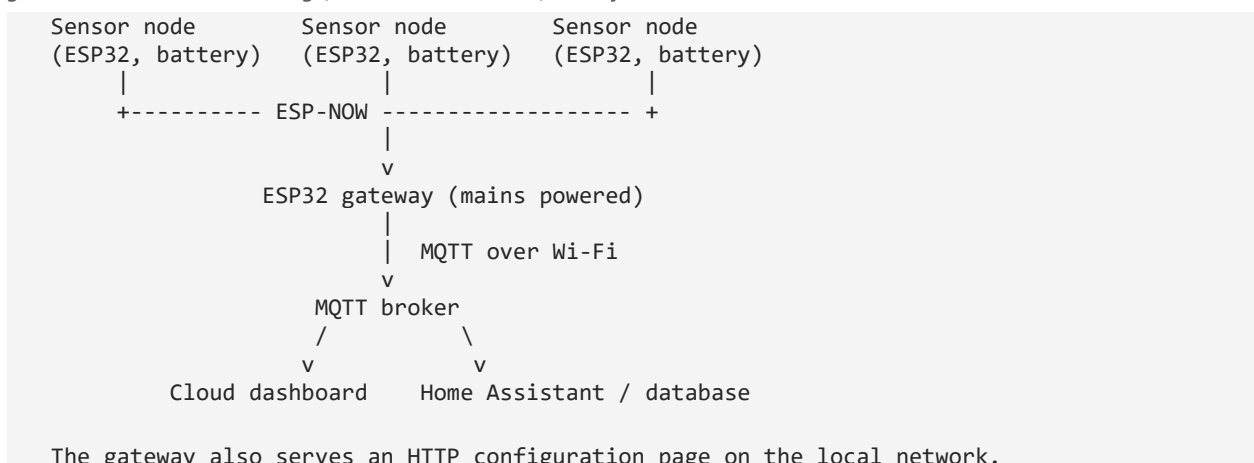
Figure 2.5: a first-pass decision path.

Does a human need to see it in a browser?	--> HTTP
Does it need to leave the building or reach a cloud service?	--> MQTT
Do more than two devices need the same data?	--> MQTT
Is it local, urgent, and between Espressif devices?	--> ESP-NOW
Is the node on a battery and asleep most of the time?	--> ESP-NOW
More than one of the above is true?	--> See Section 2.7

2.7 The Hybrid Pattern, Which Is Usually the Real Answer

Presenting the three protocols as competitors is a useful teaching device and a poor architecture. In production systems they are layers, not alternatives, and the most common serious design uses all three at once.

Figure 2.6: ESP-NOW at the edge, MQTT in the middle, HTTP for the human.



Each protocol does the job it is actually good at. The battery nodes never associate with an access point, so they survive on a coin cell. The gateway, which has mains power and can afford to hold a Wi-Fi connection open, translates between the local radio and the IP world. The broker fans the data out to whatever wants it. HTTP handles the one interaction where a person is present.

WATCH FOR THIS

The channel constraint from Section 2.3.2 lands squarely on the gateway: it is the one device that is both an ESP-NOW peer and an access-point client, so it is the device whose channel is dictated by the router while its sensors are expecting something else.

The three workable answers are to fix the access point to a known channel and configure the sensors to match, to have the sensors discover and follow the gateway's channel, or to split the roles across two ESP32s connected by UART, which is the most robust option and the one worth choosing when the link is safety-related.

2.8 Applied to the Motor Protection and Interlock Projects

Mapping this onto the project work in the companion report makes the choice concrete. The simple motor protection system has three distinct communication needs, and they do not want the same protocol.

Requirement	Protocol	Reasoning
Emergency stop and interlock between the two boards	ESP-NOW	Latency is a safety property, and the link must survive a router or internet outage. No association, no DHCP, no broker in the path.
Reporting stall events, run hours, and current readings for logging	MQTT	Small, continuous, many-to-one telemetry. Retained messages let a dashboard show the current motor state immediately, and Last Will announces a dead controller without polling.
Setting the stall current threshold and the trip delay	HTTP	A one-off interaction with a person present, served as a local configuration page. No advantage to a broker for something used twice a year.

Table 2.7: protocol allocation for the motor protection system.

The interlock payload is where Section 1 and Section 2 meet. It is a handful of fields, it must fit comfortably inside 250 bytes, and it is parsed by a receiver that sees nothing but bytes:

Listing 2.2: a scoped enum, a packed struct, and a size contract, in one ESP-NOW payload.

```
enum class MotorState : uint8_t { IDLE, RUNNING, EMERGENCY_STOP, FAULT };

struct __attribute__((packed)) InterlockFrame
{
    uint8_t    nodeId;           // offset 0
    MotorState state;            // offset 1, exactly one byte
    uint16_t   currentMa;        // offset 2
    uint32_t   uptimeMs;         // offset 4
};
static_assert(sizeof(InterlockFrame) == 8, "Interlock frame layout changed");

// 8 bytes, well within the 250-byte ESP-NOW v1.0 limit, with every byte
// at an offset both ends agree on. The receiver still validates the state
// with the decoder from Listing 1.8 before acting on it.
```

2.9 Common Mistakes Checklist

1. Sending an unpacked struct over ESP-NOW and assuming the receiver reads it correctly. Pad bytes are invisible until the fields are wrong.
2. Forgetting the ESP-NOW channel constraint on a device that also joins an access point. The symptom is a link that works on the bench and dies on site.
3. Treating the ESP-NOW send callback as proof the peer acted on the message. It confirms MAC-layer reception, not application-level processing.

4. Polling an HTTP endpoint every second on a battery device. This is the classic reason a node that should last months lasts days.
5. Using QoS 2 everywhere in MQTT "to be safe". It multiplies round trips for a guarantee most telemetry does not need.
6. Publishing to ad-hoc topic names invented per device. Agree a topic hierarchy before the second device is built.
7. Assuming a broker is optional infrastructure. It needs the same uptime, security, and backup thinking as any other server.
8. Sending ESP-NOW broadcasts and expecting them to be encrypted. Broadcast and multicast frames cannot be.

2.10 Section 2 Recap

- HTTP is a stateless request and response protocol over TCP/IP, universally supported and easy to debug, but with heavy per-request overhead and no ability for the server to push, which forces polling.
- MQTT is a publish and subscribe protocol over TCP/IP built around a broker, with a fixed header from two bytes, QoS levels, retained messages, Last Will and Testament, and persistent sessions.
- ESP-NOW skips the IP stack entirely and sends vendor-specific 802.11 action frames directly between Espressif devices, with 250 bytes of payload in v1.0 and 1,470 in v2.0.
- ESP-NOW supports up to 20 paired peers, no more than 17 of them encrypted, using CCMP with a 16-byte PMK and a per-peer 16-byte LMK; broadcast frames cannot be encrypted.
- HTTP is cheaper for a single request; MQTT wins as soon as messages repeat over one connection; ESP-NOW is in a different latency and power category because it has no association or handshake to perform.
- ESP-NOW peers must share a Wi-Fi channel, which is dictated by the access point on any device that is also a Wi-Fi client. This is the most common practical failure in mixed designs.
- The three are complementary rather than competing: ESP-NOW at the battery edge, an ESP32 gateway in the middle, MQTT to the broker and the cloud, HTTP wherever a person is looking at a screen.
- Because ESP-NOW carries raw bytes with no serialisation layer, the packed structs, fixed-width enums, and size assertions from Section 1 are what make the payload correct.

References

Primary documentation

- Espressif Systems. ESP-NOW, ESP-IDF Programming Guide (ESP32). https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/network/esp_now.html
- Espressif Systems. ESP-NOW, ESP-FAQ, Application Solutions. <https://docs.espressif.com/projects/esp-faq/en/latest/application-solution/esp-now.html>
- Espressif Systems. GCC, Migration Guide from ESP-IDF 4.x to 5.0 (unaligned access to packed struct members). <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/migration-guides/release-5.x/5.0/gcc.html>
- Espressif Systems. arduino-esp32, ESP_NOW_Network example (use of `__attribute__((packed))` on the message struct). https://github.com/espressif/arduino-esp32/blob/master/libraries/ESP_NOW/examples/ESP_NOW_Network/ESP_NOW_Network.ino
- Espressif Systems. Unaligned access to PSRAM fails with LoadStoreAlignment, issue 8295. <https://github.com/espressif/esp-idf/issues/8295>
- OASIS. MQTT Version 5.0, OASIS Standard (fixed header and Remaining Length encoding). <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- cppreference. Enumeration declaration, C++ (scoped and unscoped enumerations, underlying type). <https://en.cppreference.com/w/cpp/language/enum>
- Microsoft Learn. Enumerations (C++), including forward declaration rules for scoped and unscoped enums. <https://learn.microsoft.com/en-us/cpp/cpp/enumerations-cpp>
- SEI CERT C++ Coding Standard. INT50-CPP: Do not cast to an out-of-range enumeration value. <https://wiki.sei.cmu.edu/confluence/spaces/cplusplus/pages/88046622/>

Protocol references and articles

- HiveMQ. MQTT vs HTTP protocols in IoT and IIoT. <https://www.hivemq.com/blog/mqtt-vs-http-protocols-in-iiot/>
- HiveMQ. MQTT Essentials: core concepts, QoS, retained messages, Last Will and Testament, persistent sessions. <https://www.hivemq.com/mqtt/>
- HiveMQ. MQTT Packets: a comprehensive guide. <https://www.hivemq.com/blog/mqtt-packets-comprehensive-guide/>
- EMQX. MQTT vs HTTP: which is better suited for the Internet of Things. <https://emqx.medium.com/mqtt-vs-http-which-is-better-suited-for-the-internet-of-things-iot-5a0e080ce0a4>
- Wireless tech in IoT: MQTT vs ESP-NOW for device communication. <https://medium.com/@ks2005755/wireless-tech-in-iiot-mqtt-vs-esp-now-for-device-communication-912f16871a58>
- Arduino. Device to device communication with ESP-NOW. <https://docs.arduino.cc/tutorials/nano-esp32/esp-now/>

Memory layout, structs, unions and enums

- Understanding memory alignment and struct padding.
<https://medium.com/@yewintt.naing/understanding-memory-alignment-and-struct-padding-a4cbf11cb9c3>
- C struct padding and alignment. <https://medium.com/@haadimdwor/c-struct-padding-and-alignment-85b2011d1f5a>
- GeeksforGeeks. C unions. <https://www.geeksforgeeks.org/c/c-unions/>
- Understanding struct and union in C: a simple guide.
<https://hasanisaeed.medium.com/understanding-struct-and-union-in-c-a-simple-guide-ee961b562bcc>
- A quick guide to enums in C++. <https://ryanflynndev.medium.com/a-quick-guide-to-enums-in-c-9e376a62bee7>
- What is the difference among structure, union, enum and class.
<https://medium.com/@anilkumarsonkar/what-is-the-difference-among-structure-union-enum-and-class-3516e7d2762d>
- GeeksforGeeks. Structures, unions and enumerations in C++.
<https://www.geeksforgeeks.org/cpp/structures-unions-and-enumerations-in-cpp/>
- Modernes C++. C++ Core Guidelines: rules for enumerations.
<https://www.modernescpp.com/index.php/c-core-guidelines-rules-for-enumerations/>